

Karpathy GPT Code Study Note

A line-by-line learning note for rebuilding a tiny GPT-style language model from scratch

1. What this project is

This code trains a character-level language model on a text file (Tiny Shakespeare). It learns the statistical pattern of the next character from previous characters. The model is a miniature GPT: token embeddings + positional embeddings + stacked transformer blocks + final language head.

- Input is a sequence of character IDs.
- Output is a probability distribution for the next character at each position.
- Training goal: reduce cross-entropy loss so predicted next characters match the real text.
- Generation goal: sample one character at a time from the learned distribution.

2. Full mental pipeline

1. Read the text and convert each character into an integer ID.
2. Split the data into train and validation sets.
3. Randomly sample batches of short context windows.
4. Embed token IDs and positions into vectors.
5. Pass vectors through repeated transformer blocks.
6. Project the final hidden states into vocabulary-sized logits.
7. Compute loss with cross entropy during training.
8. Use softmax + multinomial sampling to generate new text.

3. Hyperparameters and what each one controls

These values decide training size, context length, model width, depth, and regularization.

- `batch_size = 64`: how many training examples are processed in parallel.
- `block_size = 256`: how many previous characters the model can look at.
- `max_iters = 5000`: total optimization steps.
- `eval_interval = 500`: how often to measure train/val loss.
- `learning_rate = 3e-4`: optimizer step size.
- `device = 'cuda'` if available else `'cpu'`: choose hardware automatically.
- `eval_iters = 200`: how many mini-batches to average when estimating loss.
- `n_embd = 384`: embedding / hidden dimension of the model.
- `n_head = 6`: number of attention heads.
- `n_layer = 6`: number of transformer blocks.
- `dropout = 0.2`: regularization to reduce overfitting.

Important relation: $\text{head_size} = \text{n_embd} // \text{n_head} = 384 // 6 = 64$. Each head works on a 64-dimensional subspace.

4. Random seed

`torch.manual_seed(1337)` makes the random sampling reproducible. That means batch selection, weight initialization, and generation behave consistently across runs.

5. Reading the dataset and building vocabulary

```
with open('input.txt', 'r', encoding='utf-8') as f:
    text = f.read()
```

```
chars = sorted(list(set(text)))
vocab_size = len(chars)
stoi = { ch:i for i,ch in enumerate(chars) }
itos = { i:ch for i,ch in enumerate(chars) }
encode = lambda s: [stoi[c] for c in s]
decode = lambda l: ''.join([itos[i] for i in l])
```

- text stores the full training corpus as one long string.
- set(text) finds every unique character that appears.
- sorted(...) fixes a stable ordering of the vocabulary.
- vocab_size is the number of distinct characters the model must predict.
- stoi means string-to-integer mapping.
- itos means integer-to-string mapping.
- encode converts text into a list of integer IDs.
- decode converts IDs back into readable characters.

This model is character-level, not word-level. That is simpler because every character becomes one token, but it also means the model must learn spelling and word structure from raw characters.

6. Train/validation split

```
data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9*len(data))
train_data = data[:n]
val_data = data[n:]
```

- encode(text) turns the whole corpus into integer token IDs.
- torch.tensor(..., dtype=torch.long) is required because embedding layers expect integer indices.
- 90% of the data is used for training.
- 10% is kept aside for validation so we can check generalization.

The validation set is never used for optimization. It is only for monitoring whether the model is actually learning patterns instead of memorizing the training text.

7. Mini-batch creation: get_batch

```
def get_batch(split):
    data = train_data if split == 'train' else val_data
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i+block_size] for i in ix])
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
    x, y = x.to(device), y.to(device)
    return x, y
```

- This function samples random chunks from the text.
- Each chunk has length block_size.
- x is the input context.
- y is the target sequence shifted by one character to the right.
- Example: if x = "Hell", then y = "ello" for the same window.

The shift-by-one design is the core of language modeling. The model sees a context and must predict the next token at every position.

Shape view: x has shape (batch_size, block_size) and y has the same shape.

8. Why the targets are shifted

Language modeling trains the model to predict the next character. At position t, the correct answer is token t+1. So the input window and target window are almost the same, but shifted by one step.

- Input tokens: c1, c2, c3, c4
- Targets: c2, c3, c4, c5
- The model learns one-step-ahead prediction for every position.

9. Loss estimation helper

```
@torch.no_grad()
def estimate_loss():
    out = {}
    model.eval()
    for split in ['train', 'val']:
        losses = torch.zeros(eval_iters)
        for k in range(eval_iters):
            X, Y = get_batch(split)
            logits, loss = model(X, Y)
            losses[k] = loss.item()
        out[split] = losses.mean()
    model.train()
    return out
```

- torch.no_grad() disables gradient tracking to save memory and speed up evaluation.
- model.eval() switches dropout and layer norm behavior to evaluation mode.
- The function averages loss over many random batches so the result is less noisy.
- model.train() switches back to training mode afterwards.

This is only for reporting. It does not update weights.

10. Self-attention head

```
class Head(nn.Module):
    def __init__(self, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))
        self.dropout = nn.Dropout(dropout)
```

- A head is one attention mechanism working in parallel with others.
- key, query, and value are learned linear projections.
- bias=False is common here because the attention math does not need extra bias terms.
- tril is a lower-triangular mask so future tokens cannot be seen.
- register_buffer stores tril inside the module without making it a trainable parameter.

- dropout regularizes the attention weights.

A buffer is saved with the model and moved to the device, but it is not updated by the optimizer.

11. How attention works inside one head

```
def forward(self, x):
    B, T, C = x.shape
    k = self.key(x)
    q = self.query(x)
    wei = q @ k.transpose(-2,-1) * k.shape[-1]**-0.5
    wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))
    wei = F.softmax(wei, dim=-1)
    wei = self.dropout(wei)
    v = self.value(x)
    out = wei @ v
    return out
```

- x has shape (B, T, C): batch, time, channels.
- k = key(x) and q = query(x) both become (B, T, head_size).
- wei = q @ k^T gives raw attention scores between every pair of positions.
- The scaling by head_size^{-0.5} stabilizes gradients; without it, dot products can grow too large.
- masked_fill puts -inf above the diagonal so future positions get zero probability after softmax.
- softmax turns scores into probabilities that sum to 1 across the time dimension.
- v = value(x) stores the information to be mixed.
- out = wei @ v gives a weighted average of past values.

The result at each time step is a mixture of earlier hidden states, not a single fixed previous character.

12. Why the triangular mask matters

Without the lower-triangular mask, the model would cheat during training by looking at future characters. The mask enforces causality, so the model can only use information that would really exist at generation time.

- Training and generation must match.
- Each position is allowed to attend only to itself and earlier positions.
- This is why GPT is called an autoregressive model.

13. Multi-head attention

```
class MultiHeadAttention(nn.Module):
    def __init__(self, num_heads, head_size):
        super().__init__()
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
        self.proj = nn.Linear(head_size * num_heads, n_embd)
        self.dropout = nn.Dropout(dropout)
```

- Multiple heads let the model look at the same sequence in different ways.
- Each head learns its own query/key/value projections.
- ModuleList registers all heads properly so PyTorch tracks their parameters.
- After concatenation, proj mixes information from all heads back into n_embd dimensions.

Different heads can specialize in different relationships, such as nearby characters, syntax-like patterns, or longer dependencies.

14. Feed-forward network

```
class FeedFoward(nn.Module):
    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout),
        )
```

- This part does not mix time steps; it works independently at each position.
- First linear layer expands the representation to 4x width.
- ReLU adds non-linearity.
- Second linear layer compresses back to n_embd.
- Dropout is applied for regularization.

Attention is for communication across tokens; the feed-forward network is for local computation inside each token representation.

15. Transformer block

```
class Block(nn.Module):
    def __init__(self, n_embd, n_head):
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedFoward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.sa(self.ln1(x))
        x = x + self.ffwd(self.ln2(x))
        return x
```

- A block is the standard GPT building unit.
- LayerNorm is applied before attention and before feed-forward (pre-norm style).
- Residual connections $x + \dots$ help gradients flow and make deep networks trainable.
- First the block performs communication via attention, then computation via feed-forward.

Residual learning means the block learns a correction to x instead of replacing x entirely. This is one reason deep transformers train much more reliably.

16. GPTLanguageModel: the full model

```
class GPTLanguageModel(nn.Module):
    def __init__(self):
        super().__init__()
```

```

self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
self.position_embedding_table = nn.Embedding(block_size, n_embd)
self.blocks = nn.Sequential(*[Block(n_embd, n_head=n_head) for _ in range(n_layer)])
self.ln_f = nn.LayerNorm(n_embd)
self.lm_head = nn.Linear(n_embd, vocab_size)
self.apply(self._init_weights)

```

- token_embedding_table learns a vector for every character ID.
- position_embedding_table learns a vector for every position in the context window.
- blocks stacks n_layer transformer blocks.
- ln_f is a final normalization layer.
- lm_head maps hidden states to vocabulary-sized logits.
- apply(self._init_weights) initializes weights with a stable distribution.

This is the full GPT stack in miniature: embeddings -> transformer blocks -> final projection to token probabilities.

17. Weight initialization

```

def _init_weights(self, module):
    if isinstance(module, nn.Linear):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if module.bias is not None:
            torch.nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Embedding):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

```

- Linear layers get small random weights.
- Biases start at zero.
- Embedding tables also get small random weights.
- Good initialization prevents unstable early training.

Without careful initialization, activations can explode or collapse, especially in deep models.

18. Forward pass of the full model

```

def forward(self, idx, targets=None):
    B, T = idx.shape
    tok_emb = self.token_embedding_table(idx)
    pos_emb = self.position_embedding_table(torch.arange(T, device=device))
    x = tok_emb + pos_emb
    x = self.blocks(x)
    x = self.ln_f(x)
    logits = self.lm_head(x)

```

- idx is a batch of integer token IDs with shape (B, T).
- tok_emb converts token IDs to learned vectors, shape (B, T, C).
- pos_emb adds position information so the model knows order, not just content.
- x = tok_emb + pos_emb combines both signals.
- The transformer blocks process the sequence.
- Final linear layer outputs logits for every token in the vocabulary.

Key point: embeddings are added, not concatenated. That keeps the representation size fixed at n_embd.

19. Why position embeddings are needed

Attention alone does not know the absolute order of tokens. If two sequences have the same tokens in different order, self-attention without position information may treat them too similarly. Position embeddings inject order information into the representation.

20. Cross-entropy loss

```
if targets is None:
    loss = None
else:
    B, T, C = logits.shape
    logits = logits.view(B*T, C)
    targets = targets.view(B*T)
    loss = F.cross_entropy(logits, targets)
```

- If targets is absent, the model only returns logits for generation.
- For training, logits are reshaped from (B, T, vocab_size) to (B*T, vocab_size).
- Targets are reshaped to (B*T,).
- cross_entropy compares each prediction with the correct next token.

Cross entropy is the standard loss for classification over the vocabulary. Each time step is a classification problem: which token comes next?

21. Generate method

```
def generate(self, idx, max_new_tokens):
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -block_size:]
        logits, loss = self(idx_cond)
        logits = logits[:, -1, :]
        probs = F.softmax(logits, dim=-1)
        idx_next = torch.multinomial(probs, num_samples=1)
        idx = torch.cat((idx, idx_next), dim=1)
    return idx
```

- Generation happens one token at a time.
- idx_cond keeps only the last block_size tokens so the context length stays bounded.
- The model predicts logits for the whole window, but only the final position is needed.
- softmax turns logits into probabilities.
- torch.multinomial samples the next token from that probability distribution.
- The sampled token is appended to the sequence and the loop repeats.

This is autoregressive sampling: every new token depends on all previously generated tokens in the current window.

22. Why multinomial sampling is used

The model could always choose the highest-probability token, but that would make outputs repetitive. Sampling preserves diversity. That is why generation is creative rather than always deterministic.

- argmax gives the single most likely token.
- multinomial samples according to probabilities.

- Sampling is usually better for text generation.

23. Model creation and parameter count

```
model = GPTLanguageModel()
m = model.to(device)
print(sum(p.numel() for p in m.parameters())/1e6, 'M parameters')
```

- model creates the neural network.
- to(device) moves the model to GPU if available.
- sum(p.numel() for p in m.parameters()) counts trainable parameters.
- Dividing by 1e6 prints the size in millions.

24. Optimizer

```
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)
```

- AdamW is Adam with decoupled weight decay.
- It is a strong default optimizer for transformer training.
- learning_rate controls how large each update step is.

25. Training loop

```
for iter in range(max_iters):
    if iter % eval_interval == 0 or iter == max_iters - 1:
        losses = estimate_loss()
        print(f'step {iter}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}')
```

```
xb, yb = get_batch('train')
logits, loss = model(xb, yb)
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```

- Every eval_interval steps, the model is evaluated on train and validation batches.
- A new batch is sampled.
- Forward pass computes logits and loss.
- zero_grad clears old gradients.
- loss.backward computes gradients through the entire network.
- optimizer.step updates the weights.

This is the standard PyTorch training pattern: forward -> loss -> backward -> step.

26. Final text generation

```
context = torch.zeros((1, 1), dtype=torch.long, device=device)
print(decode(m.generate(context, max_new_tokens=500)[0].tolist()))
```

- context starts with one token of 0, which is just an initial seed.
- generate extends that seed into a long sequence.
- decode converts the sampled token IDs back into characters.

27. Common confusions

- Why use long dtype? Because embeddings need integer token indices, not floats.
- Why add token and position embeddings? Because the model needs both identity and order.
- Why does attention use $q @ k^T$? Because each query measures similarity to all keys.
- Why is the mask lower triangular? Because future information is forbidden.
- Why reshape for cross entropy? Because PyTorch expects a 2D logits matrix and 1D target vector.
- Why only use logits[:, -1, :] in generation? Because we only need the next token after the current context.

28. What each module is responsible for

- Embedding layers: turn IDs into vectors.
- Attention heads: let tokens communicate with earlier tokens.
- Multi-head wrapper: combine several communication patterns.
- Feed-forward network: transform each token independently.
- Residual connections: keep training stable.
- LayerNorm: stabilize activations.
- LM head: convert hidden vectors into vocabulary logits.

29. How to rebuild this from memory

9. Start with a text file and build stoi/itos.
10. Turn the full text into a tensor of token IDs.
11. Make random x/y windows where y is shifted by one.
12. Create token and position embeddings.
13. Stack transformer blocks with attention, feed-forward, LayerNorm, and residual connections.
14. Project the final hidden states to vocabulary logits.
15. Train with cross entropy and AdamW.
16. Generate by repeatedly sampling the next token.

If you can reconstruct those 8 steps without looking, you understand the code at architecture level.

30. Debug checklist

- Check tensor shapes at every stage.
- Make sure x and y are shifted by exactly one token.
- Make sure the model is moved to the same device as the data.
- Check that masked attention is actually causal.
- Check that logits are flattened correctly before cross entropy.
- Check that generation uses only the last time step before sampling.

31. Final takeaway

This code is not just a program to run once. It is a blueprint for a transformer language model. The real goal is to learn the pattern: embeddings -> attention -> feed-forward -> residuals -> normalization -> prediction -> sampling. Once that pattern is clear, Karpathy-style code, GPT papers, and many modern transformer implementations become much easier to read and rebuild.